

 LSTM_26 (1).ipynb

Act as an expert Senior Python & Machine Learning Instructor. I have attached a Jupyter Notebook (`.ipynb`)

I do not want high-level summaries or generic documentation. Instead, please provide an exhaustive, line-by-line, mechanical breakdown of every single operational line of code in this file.

For EVERY line of code (especially the mathematical steps, array manipulations, and plotting configurations), please address the following structural points:

1. UNDERLYING MECHANICS & ARGUMENTS: What do the specific function parameters or arguments mean at a low level? (For example, if a line reads `axis=0`, `full_matrices=False`, or `::-1`, explain exactly what that specific configuration tells the computer to do step-by-step).
2. STRUCTURAL & SHAPE TRANSFORMATION: What are the input array shapes and data types before this line executes, and what is the exact output shape/data type immediately after it executes? (Explicitly use matrix dimensions like (N, D), (D, D), etc., to show how data is being stretched, collapsed, or transformed).
3. BEHAVIORAL ALTERNATIVES: What would happen to the code's behavior, execution flow, data shapes, and logic if those specific parameters were changed to other valid/invalid states? (Provide counter-examples of alternative parameter values and explain why they would either alter the outcome or break the script).
4. ALGORITHMIC & MATH JUSTIFICATION: Why is this exact configuration mathematically or structurally mandatory for the algorithm to function properly?

Please maintain this deep, parameter-focused engineering perspective throughout your entire line-by-line response.

Cell 1: Data Generation & Preprocessing Libraries

Line 1: `import numpy as np`

- 1. Underlying Mechanics & Arguments:** This statement loads the `numpy` library into the runtime environment and maps its namespace to the alias `np`. Mechanically, Python searches `sys.path` for the compiled C-extensions and modules comprising NumPy, initializes the top-level package, and creates a reference within the global symbol table.
- 2. Structural & Shape Transformation:** There are no array shape transformations. The global namespace receives an object reference `<module 'numpy' from ...>` of type `module`.
- 3. Behavioral Alternatives:** Changing this to `import numpy` without the alias requires all subsequent mathematical calls to be fully qualified (e.g., `numpy.arange`). Omitting the statement results in a `NameError: name 'np' is not defined` when execution reaches any mathematical operations.
- 4. Algorithmic & Math Justification:** Mandatory for high-performance vectorization. NumPy wraps underlying C array structures (`ndarray`), ensuring contiguous memory allocation and bypassing Python's reference-counting loop overhead during synthetic data synthesis.

Line 2: `import pandas as pd`

- 1. Underlying Mechanics & Arguments:** This statement initializes the `pandas` library, mapping it to the namespace alias `pd`. It constructs a high-level wrapper around NumPy arrays to handle tabular, heterogeneously typed structures and time-series indexes.
- 2. Structural & Shape Transformation:** No direct data transformations occur. It adds a reference to `<module 'pandas' from ...>` into the local environment.
- 3. Behavioral Alternatives:** If imported as `import pandas`, downstream calls to `pd.DataFrame` must be modified to `pandas.DataFrame`. Omitting this will break the script with a `NameError` at the DataFrame instantiation step.
- 4. Algorithmic & Math Justification:** Required here specifically for its calendar-aware indexing (`pd.date_range`). This provides structural temporal validation, alignment, and formatting for sequence generation.

Line 3: `import matplotlib.pyplot as plt`

- 1. Underlying Mechanics & Arguments:** This statement imports the state-based plotting interface `pyplot` contained inside the `matplotlib` package, aliasing it as `plt`. It hooks into the active graphic rendering engine (backend) configured for the runtime environment.

2. **Structural & Shape Transformation:** Namespace registration only; registers a `<module 'matplotlib.pyplot' from ...>` reference.
3. **Behavioral Alternatives:** Using alternative backends explicitly via `matplotlib.use('Agg')` would suppress GUI rendering and write files directly to disk. Omitting this step results in a `NameError` during evaluation canvas construction.
4. **Algorithmic & Math Justification:** Structurally necessary to build and display loss curves or inference tracking plots. It maps numerical ndarray data structures directly to graphic coordinate spaces.

Line 4: `from sklearn.preprocessing import MinMaxScaler`

1. **Underlying Mechanics & Arguments:** This extracts the `MinMaxScaler` class definition from Scikit-Learn's `preprocessing` submodule. It avoids loading the entire package namespace into memory, isolating only the class responsible for min-max bounding transformations.
2. **Structural & Shape Transformation:** Instantiates the type reference `MinMaxScaler` within the runtime environment's global symbol table.
3. **Behavioral Alternatives:** Replacing this with `import sklearn` forces long namespaces like `sklearn.preprocessing.MinMaxScaler()`. Omitting this prevents data scaling, crashing execution at the instantiation line.
4. **Algorithmic & Math Justification:** Crucial for Recurrent Neural Networks (RNNs). Because LSTMs utilize sigmoid and hyperbolic tangent (`tanh`) activation functions, unscaled inputs with high variance cause gradient saturation (vanishing gradients).

Line 5: `from tensorflow.keras.models import Sequential`

1. **Underlying Mechanics & Arguments:** Imports the `Sequential` class from the Keras API embedded inside TensorFlow. Mechanically, a `Sequential` object represents a linear, single-input, single-output stack of layer instances implemented as a directed acyclic graph (DAG) of tensor operations.
2. **Structural & Shape Transformation:** Registers the class constructor reference `Sequential` in the local scope.
3. **Behavioral Alternatives:** Using the alternative Keras Functional API (`from tensorflow.keras import Model`) would change how layers are linked, switching from `.add()` syntax to explicit tensor calls: `outputs = LSTM()(inputs)`. Omitting this prevents network structural instantiation.
4. **Algorithmic & Math Justification:** Mandated for compiling sequential structural topologies, automating layer-to-layer shape inference during forward propagation.

Line 6: `from tensorflow.keras.layers import LSTM, Dense, Dropout`

1. **Underlying Mechanics & Arguments:** Selectively imports layer definitions from the Keras layers module: `LSTM` (Long Short-Term Memory recurrent cell blocks), `Dense` (regular deeply connected layer), and `Dropout` (stochastic zeroing tensor mask).
2. **Structural & Shape Transformation:** Adds type identifiers for `LSTM`, `Dense`, and `Dropout` layers to the module workspace.
3. **Behavioral Alternatives:** Omitting any single layer component will trigger a `NameError` during architecture compilation. Replacing them with custom layers requires subclassing `tf.keras.layers.Layer` and implementing explicit tensor transformations.
4. **Algorithmic & Math Justification:** These classes contain the structural equations for processing temporal dependencies, linearly projecting dimensional states, and applying stochastic regularizers.

Line 8: `def generate_weather_data(days=1000):`

1. **Underlying Mechanics & Arguments:** Declares a named function block with a default integer parameter `days=1000`. The python interpreter parses the code block, validates syntax correctness, and binds the block to the variable identifier `generate_weather_data`.
2. **Structural & Shape Transformation:** Creates a callable object function reference inside memory space.
3. **Behavioral Alternatives:** If the default argument is altered to a non-integer type (e.g., `days='1000'`), downstream numerical range operations will throw a `TypeError`.
4. **Algorithmic & Math Justification:** Encourages structural encapsulation, isolating raw synthetically generated variables from feature engineering pipelines.

Line 9: `np.random.seed(42)`

1. **Underlying Mechanics & Arguments:** Sets the internal state of NumPy's legacy Mersenne Twister pseudo-random number generator to an arbitrary integer seed value of `42`.
2. **Structural & Shape Transformation:** None. Modifies the internal registers of the global `numpy.random` state object.
3. **Behavioral Alternatives:** Omitting this statement or changing the seed to another integer value alters the deterministic stream of pseudo-random floats generated on Line 12. This leads to slightly different synthetic weather distributions across execution runs.
4. **Algorithmic & Math Justification:** Mathematically mandatory to guarantee execution idempotency. It ensures that the pseudo-random noise added to the underlying sine wave is identical across different environments, preserving reproducible debug trials.

Line 11: `time = np.arange(0, days)`

1. **Underlying Mechanics & Arguments:** Invokes `numpy.arange(start, stop)` to populate a 1D sequence array. It increments by a default step value of `1` from inclusive low index `0` up to

exclusive upper bound `days` (1000).

2. **Structural & Shape Transformation:** Outputs a 1D NumPy ndarray with data type `int64` and shape `(1000,)`.
3. **Behavioral Alternatives:** Setting a step argument like `np.arange(0, days, 0.5)` changes the output shape to `(2000,)`, changing the underlying sampling rate of the downstream sine function. Passing a string format will throw a `TypeError`.
4. **Algorithmic & Math Justification:** Provides the monotonic temporal axis ($t \in \mathbb{N}_0$) necessary to compute continuous periodic wave transformations across discrete time steps.

Line 12: `temp = 20 + 10 * np.sin(2 * np.pi * time / 365) + np.random.normal(0, 2, days)`

1. **Underlying Mechanics & Arguments:** Performs vectorized arithmetic calculation.
 - `2 * np.pi` : Multiplies scalar constant 2 by floating scalar $\pi \approx 3.14159$, creating the angular frequency scalar value.
 - `* time / 365` : Multiplies the frequency value by every integer in the `time` array, then divides by the scalar calendar year value 365. This maps the domain from day step offsets to radians, setting a structural seasonal periodicity of 365 steps.
 - `np.sin(...)` : Computes the element-wise sine vector.
 - `20 + 10 * ...` : Scales the output magnitude by a factor of 10 (amplitude) and shifts it upwards by 20 (mean base offset temperature).
 - `np.random.normal(0, 2, days)` : Generates a 1D array of Gaussian noise with a mean (μ) of 0, a standard deviation (σ) of 2, and a length equal to `days` (1000).

2. Structural & Shape Transformation:

- Input `time` shape: `(1000,)` with type `int64`.
- `np.random.normal` shape: `(1000,)` with type `float64`.
- Output `temp` shape: `(1000,)` with type `float64`, resulting from element-wise vector addition.

3. **Behavioral Alternatives:** Increasing the noise standard deviation to `20` completely washes out the underlying seasonal sine wave structure, resulting in a low signal-to-noise ratio that makes it difficult for the LSTM to learn. Removing the `time / 365` normalization step results in rapid, chaotic oscillations across small step intervals.
4. **Algorithmic & Math Justification:** Mathematically models a composite real-world time series containing a predictable seasonal baseline combined with stochastic environmental noise:

$$y(t) = 20 + 10 \sin\left(\frac{2\pi t}{365}\right) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 2^2)$$

Line 14: `df = pd.DataFrame({`

1. **Underlying Mechanics & Arguments:** Initializes a new `pd.DataFrame` instance using a key-value dictionary structure. The keys serve as structural string labels for the columns, while the values hold equal-length array data sequences.
2. **Structural & Shape Transformation:** No direct array manipulation yet; initiates structural allocation for a 2-dimensional tabular entity.
3. **Behavioral Alternatives:** Passing arrays of mismatching lengths will immediately raise a `ValueError`: All arrays must be of the same length .
4. **Algorithmic & Math Justification:** Provides structured access to columns via text labels rather than raw integer matrix indices, preventing downstream layout indexing errors.

Line 15: `'date': pd.date_range(start='2020-01-01', periods=days, freq='D'),`

1. **Underlying Mechanics & Arguments:** Generates an array of absolute datetime timestamps starting at Gregorian calendar reference date `'2020-01-01'` . The total count is bound by `periods=days` (1000), incrementing daily based on the frequency parameter `freq='D'` .
2. **Structural & Shape Transformation:** Produces a `DatetimeIndex` array of type `datetime64[ns]` with a 1D shape of `(1000,)`.
3. **Behavioral Alternatives:** Changing `freq='D'` to `freq='H'` shifts temporal sampling from daily steps to hourly intervals, altering the underlying context of the seasonal window parameters.
4. **Algorithmic & Math Justification:** Establishes a true calendar timeline, providing explicit validation for temporal consistency and helping prevent out-of-order sequence indexing.

Line 16: `'temperature': temp`

1. **Underlying Mechanics & Arguments:** Associates the `'temperature'` dictionary key string directly with the previously computed `temp` array vector.
2. **Structural & Shape Transformation:** Binds the 1D `float64` array of shape `(1000,)` into the data dictionary structure.
3. **Behavioral Alternatives:** If `temp` is modified or sliced to a shorter length beforehand, execution breaks during DataFrame assembly due to dimension mismatch.
4. **Algorithmic & Math Justification:** Pairs continuous target values with their corresponding calendar index positions for subsequent feature processing.

Line 17: `})`

1. **Underlying Mechanics & Arguments:** Closes the column-mapping dictionary structure, passing it directly into the inner memory allocation routines of the `pd.DataFrame` constructor.
2. **Structural & Shape Transformation:** Transforms individual dictionary sequences into a single cohesive 2D structural layout of shape `(1000, 2)`.

3. **Behavioral Alternatives:** Missing this bracket triggers a basic Python parsing syntax error (`SyntaxError: unexpected EOF while parsing`).
4. **Algorithmic & Math Justification:** Completes structural table assembly, constructing the columnar underlying memory blocks.

Line 19: `return df`

1. **Underlying Mechanics & Arguments:** Evaluates the local scope variable identifier `df` and returns its object pointer reference back to the outer environment calling context, cleaning up the function's internal call stack frame.
2. **Structural & Shape Transformation:** The returned value is a 2D `DataFrame` structure of shape `(1000, 2)`.
3. **Behavioral Alternatives:** Omitting this statement causes the function to implicitly return `None` . This breaks line 21 when attempting to call method handles on a non-existent object reference.
4. **Algorithmic & Math Justification:** Passes clean, isolated, and formatted feature schemas out of localized calculation functions.

Line 21: `df = generate_weather_data()`

1. **Underlying Mechanics & Arguments:** Executes the function compiled on Line 8 with default parameters, assigning the returned `DataFrame` reference to the global variable identifier `df` .
2. **Structural & Shape Transformation:** Instantiates a 2D global `DataFrame` variable of shape `(1000, 2)`, containing columns `date` (`datetime64[ns]`) and `temperature` (`float64`).
3. **Behavioral Alternatives:** Passing an explicit value like `generate_weather_data(days=500)` modifies the internal steps, altering downstream data splitting matrix slices.
4. **Algorithmic & Math Justification:** Generates the primary data table used for feature scaling and sequence generation.

Line 22: `print(df.head())`

1. **Underlying Mechanics & Arguments:** The `.head()` method extracts the leading 5 rows from the `DataFrame`. The resulting subset is then written to the standard output stream (`stdout`) via the `print()` function.
2. **Structural & Shape Transformation:** Slices the original `(1000, 2)` `DataFrame` to produce a temporary visualization slice of shape `(5, 2)`.
3. **Behavioral Alternatives:** Changing to `df.head(50)` expands the console display layout to 50 rows. Omitting this step suppresses visual data validation but has no effect on underlying matrix transformations.
4. **Algorithmic & Math Justification:** Serves as an empirical debugging step to verify correct data synthesis, temporal frequency generation, and baseline values before triggering deep learning

Cell 2: Feature Scaling & Training Data Formatting

Line 1: `scaler = MinMaxScaler(feature_range=(0, 1))`

1. **Underlying Mechanics & Arguments:** Instantiates the Scikit-Learn transformer object. The parameter `feature_range=(0, 1)` defines the explicit bounding box minimum and maximum target thresholds for the transformation.
2. **Structural & Shape Transformation:** Instantiates a stateful internal object tracking structure. No dataset matrix transformation occurs here.
3. **Behavioral Alternatives:** Altering the target boundaries to `feature_range=(-1, 1)` changes the scaled data range to match a `tanh` activation layer function. Omitting the parameter leaves the default range at `(0, 1)`.
4. **Algorithmic & Math Justification:** Initializes the tracking variables (x_{min}, x_{max}) required for min-max scaling:
$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \cdot (max - min) + min$$

Line 2: `scaled_data = scaler.fit_transform(df['temperature'].values.reshape(-1, 1))`

1. **Underlying Mechanics & Arguments:** This line chains multiple matrix transformation operations together:
 - `df['temperature']` : Extracts a 1D Series wrapper from the DataFrame structure.
 - `.values` : Unwraps the series index metadata, exposing the underlying raw NumPy ndarray pointer array.
 - `.reshape(-1, 1)` : Changes the array dimensions. The value `-1` acts as a dynamic wildcard, telling the engine to calculate the remaining row axis length automatically based on the total elements, while fixing the second dimension to exactly `1` column.
 - `scaler.fit_transform(...)` : Computes the data min-max statistics on the reshaped array, updates its internal tracking states, and applies the min-max formula.
2. **Structural & Shape Transformation:**
 - Raw Series shape: `(1000, )`
 - `.reshape(-1, 1)` output shape: `(1000, 1)` with data type `float64`.
 - `fit_transform` output shape: `(1000, 1)` with data type `float64`.
3. **Behavioral Alternatives:** Skipping the `.reshape(-1, 1)` call passes a 1D array of shape `(1000, )` to the scaler, which triggers a `ValueError` because Scikit-Learn transformers expect a 2D sample/feature matrix.
4. **Algorithmic & Math Justification:** Converts data structures into standard matrix format (Samples, Features). Scaling feature columns prevents exploding or vanishing gradients during

backpropagation through time (BPTT).

[Image diagram showing a 1D array of shape (1000,) being reshaped into a 2D column matrix of shape (1000,1) for compliance with Scikit-Learn estimators]

Line 4: `def create_sequences(data, seq_length):`

1. **Underlying Mechanics & Arguments:** Declares a sequence generation utility. `data` is the normalized array, and `seq_length` specifies the temporal sliding window size used for forecasting.
2. **Structural & Shape Transformation:** Registers the function name reference `create_sequences` inside the current active namespace.
3. **Behavioral Alternatives:** Defining this without explicit arguments or passing non-indexable scalar types will break internal array slicing and throw a `TypeError`.
4. **Algorithmic & Math Justification:** Encapsulates the sliding window matrix operations required to convert continuous time series data into overlapping sequences suitable for supervised learning.

Line 5: `x, y = [], []`

1. **Underlying Mechanics & Arguments:** Uses Python's unpacking syntax to initialize two empty standard lists in memory, mapping them to the variables `x` (features) and `y` (targets).
2. **Structural & Shape Transformation:** Allocates two empty list objects (`[]`) with an initial length of 0.
3. **Behavioral Alternatives:** Initializing these as predefined scalar zeros (e.g., `x = 0`) causes an `AttributeError` when attempting to call `.append()` later in the loop.
4. **Algorithmic & Math Justification:** Allocates dynamic memory buffers to store sequence chunks before stacking them into dense contiguous arrays.

Line 6: `for i in range(len(data) - seq_length):`

1. **Underlying Mechanics & Arguments:** Sets up a loop using an index tracker `i` that iterates over a specified range. The upper bound is calculated as `len(data) - seq_length`, which subtracts the window length from the total data points ($1000 - 30 = 970$). This prevents the slicing operation from running past the end of the array.
2. **Structural & Shape Transformation:** Loops through 970 iterations, generating sequential scalar index integers from `0` to `969`.
3. **Behavioral Alternatives:** Forgetting to subtract `seq_length` (e.g., `range(len(data))`) causes the trailing loop steps to slice empty segments, resulting in partial inputs or index errors.
4. **Algorithmic & Math Justification:** Sets the total sample size for the sequence dataset, ensuring that every input window has a valid, corresponding future target label.

Line 7: `x.append(data[i:i + seq_length])`

1. **Underlying Mechanics & Arguments:** Slices a continuous block of rows from the data matrix starting at index position `i` up to exclusive boundary `i + seq_length`. This 30-row slice is appended directly into the `x` list container.
2. **Structural & Shape Transformation:** Extracts an array slice of shape $(30, 1)$ from the $(1000, 1)$ source data array during each iteration, appending it as a new element to list `x`.
3. **Behavioral Alternatives:** Modifying the slice index to `data[i]` appends individual scalars rather than sequential history blocks, which removes the temporal dimension needed for the LSTM input layer.
4. **Algorithmic & Math Justification:** Builds the past historical window matrix required to train recurrent models on sequential dependencies.

Line 8: `y.append(data[i + seq_length])`

1. **Underlying Mechanics & Arguments:** Extracts the feature scalar value located exactly at index position `i + seq_length` (the immediate next step following the historical window) and appends it to list `y`.
2. **Structural & Shape Transformation:** Extracts an individual value slice of shape $(1,)$ from the source data array during each iteration, appending it as a new element to list `y`.
3. **Behavioral Alternatives:** If shifted to `i + seq_length + 1`, the network shifts to forecasting two steps into the future, creating a temporal mismatch if the labels are not explicitly aligned.
4. **Algorithmic & Math Justification:** Pairs each historical sequence window with its immediate next chronological value, formulating time series forecasting as a supervised learning task.

[Image diagram of a sliding window process converting a 1D time series into input sequences X of length 30 and corresponding single-step target outputs y]

Line 9: `return np.array(x), np.array(y)`

1. **Underlying Mechanics & Arguments:** Converts the native Python lists of array slices into dense, multi-dimensional NumPy arrays using `np.array()`. This consolidates fragmented memory references into contiguous memory blocks.
2. **Structural & Shape Transformation:**
 - Input list `x` containing 970 slices of shape $(30, 1)$ → transformed into a 3D tensor of shape $(970, 30, 1)$ with data type `float64`.
 - Input list `y` containing 970 slices of shape $(1,)$ → transformed into a 2D tensor of shape $(970, 1)$ with data type `float64`.
3. **Behavioral Alternatives:** Returning raw, un-vectorized Python lists causes an immediate error when passed to TensorFlow, which requires structured tensors or NumPy arrays for graph

initialization.

4. **Algorithmic & Math Justification:** Converts sequence data into the standard 3D tensor format required by Keras LSTM layers: (Samples, Time Steps, Features) .

Line 11: `SEQ_LENGTH = 30`

1. **Underlying Mechanics & Arguments:** Defines a global integer constant named `SEQ_LENGTH` and assigns it a value of `30` . This sets the historical lookback window size for the model.
2. **Structural & Shape Transformation:** Allocates an immutable integer object reference within the global workspace.
3. **Behavioral Alternatives:** Setting `SEQ_LENGTH = 1` reduces the lookback window to a single step, which removes the LSTM's ability to learn long-term cyclical or seasonal dependencies.
4. **Algorithmic & Math Justification:** Sets the input sequence length, defining how many historical time steps the model looks back to predict the next value.

Line 13: `X, y = create_sequences(scaled_data, SEQ_LENGTH)`

1. **Underlying Mechanics & Arguments:** Invokes the sequence generation function with the normalized data and lookback window size, unpacking the returned arrays into global variables `X` and `y` .
2. **Structural & Shape Transformation:**
 - `X` becomes a 3D NumPy array of shape `(970, 30, 1)` with data type `float64` .
 - `y` becomes a 2D NumPy array of shape `(970, 1)` with data type `float64` .
3. **Behavioral Alternatives:** Reversing the variable unpacking order (`y, X = ...`) causes shape mismatches down the line, as the model will attempt to treat the target vector as input features.
4. **Algorithmic & Math Justification:** Creates the complete, structural supervised dataset required for model training and evaluation.

Line 15: `train_size = int(len(X) * 0.8)`

1. **Underlying Mechanics & Arguments:** Computes the training split index by multiplying the total samples by `0.8` ($970 \times 0.8 = 776.0$). The result is converted to an integer via `int()` to make it a valid index slice position.
2. **Structural & Shape Transformation:** Creates a scalar integer variable `train_size` with a value of `776` .
3. **Behavioral Alternatives:** Removing the `int()` wrapper leaves a floating-point value (`776.0`), which will trigger a `TypeError` when used as an index slice for arrays.
4. **Algorithmic & Math Justification:** Determines the temporal split point for the dataset. Using a chronological split rather than a random shuffle preserves the contiguous sequence order, preventing data leakage from the future into the past.

Line 17: `X_train, X_test = X[:train_size], X[train_size:]`

1. **Underlying Mechanics & Arguments:** Standard Python slice syntax splits the 3D tensor `X` into two separate arrays along its first dimension (samples), using `train_size` (776) as the split point.
2. **Structural & Shape Transformation:**
 - `X[:train_size]` extracts rows from index 0 to 775, outputting a 3D tensor of shape (776, 30, 1).
 - `X[train_size:]` extracts rows from index 776 to 969, outputting a 3D tensor of shape (194, 30, 1).
3. **Behavioral Alternatives:** Shuffling the indices before slicing removes the chronological sequence order, causing temporal leakage and invalidating backpropagation assumptions.
4. **Algorithmic & Math Justification:** Divides input features into training and evaluation sets while maintaining chronological order across the temporal split point.

Line 18: `y_train, y_test = y[:train_size], y[train_size:]`

1. **Underlying Mechanics & Arguments:** Splits the 2D target label array `y` along its first dimension using the same `train_size` index (776).
2. **Structural & Shape Transformation:**
 - `y_train` receives a 2D array slice of shape (776, 1) with data type `float64`.
 - `y_test` receives a 2D array slice of shape (194, 1) with data type `float64`.
3. **Behavioral Alternatives:** Slicing the target labels with a different split point than the features (e.g., `train_size + 5`) pairs inputs with the wrong chronological targets, misaligning the supervised training setup.
4. **Algorithmic & Math Justification:** Segregates target labels into training and testing sets, ensuring they align perfectly with the corresponding input feature sequences.

Line 20: `print(f"Training shape: {X_train.shape}, Testing shape: {X_test.shape}")`

1. **Underlying Mechanics & Arguments:** Formats a string containing the structural dimension properties of the split tensors and prints the output to the console window.
2. **Structural & Shape Transformation:** No data transformations occur; displays shape strings to standard output.
3. **Behavioral Alternatives:** Omitting this statement suppresses console confirmation but does not alter execution flow or data shapes.
4. **Algorithmic & Math Justification:** Validates tensor dimensions before initializing the model, confirming that the data matches the expected input shapes for the network layers.

Cell 3: Deep Neural Network Architecture & Model Compilation

Line 1: `model = Sequential([`

1. **Underlying Mechanics & Arguments:** Initializes a Keras `Sequential` model container object. It accepts a linear list of individual layer components, linking them sequentially into an internal network execution graph.
2. **Structural & Shape Transformation:** Instantiates an empty sequential model tracking structure in memory.
3. **Behavioral Alternatives:** Passing layers out of order or using non-layer objects inside the list breaks initialization and throws a `TypeError`.
4. **Algorithmic & Math Justification:** Simplifies the construction of linear feed-forward networks by managing internal layer tensor dependencies automatically.

Line 2: `LSTM(50, return_sequences=True, input_shape=(SEQ_LENGTH, 1)),`

1. **Underlying Mechanics & Arguments:** Adds a stateful Long Short-Term Memory (LSTM) layer containing 50 hidden recurrent units.
 - `50` : Specifies the dimensionality of the internal hidden state vector (h_t) and cell state vector (c_t).
 - `return_sequences=True` : Tells the layer to output its hidden state at every individual time step in the input sequence, rather than just the final step.
 - `input_shape=(SEQ_LENGTH, 1)` : Defines the shape of the incoming data windows, specifying 30 time steps with 1 feature per step.
2. **Structural & Shape Transformation:**
 - Input tensor shape: `(None, 30, 1)`, where `None` represents a dynamic batch size.
 - Output tensor shape: `(None, 30, 50)`, matching the 50 hidden states generated across all 30 time steps.
3. **Behavioral Alternatives:** Setting `return_sequences=False` forces the layer to output only the hidden state of the final time step, reducing the output tensor shape to `(None, 50)`. This will break the downstream LSTM layer, which expects a 3D sequential input.
4. **Algorithmic & Math Justification:** Stacking recurrent layers requires setting `return_sequences=True` so that downstream recurrent layers receive the necessary 3D sequence tensors. This enables the model to extract complex temporal features across different levels of abstraction.

Line 3: `Dropout(0.2),`

1. **Underlying Mechanics & Arguments:** Instantiates a regularizing layer wrapper. The parameter `0.2` defines a 20% probability of stochastically masking elements. During each training forward

pass, it randomly sets 20% of the incoming tensor elements to zero.

2. Structural & Shape Transformation:

- Input tensor shape: `(None, 30, 50)` .
- Output tensor shape: `(None, 30, 50)` . Dimensions are preserved, but a stochastically selected 20% of the data values are zeroed out, while the remaining values are scaled by $\frac{1}{1-0.2} = 1.25$ to maintain total energy balance.

3. **Behavioral Alternatives:** Setting the rate to `0.9` drops 90% of feature information, causing underfitting. Setting it to a value outside the valid range `[0, 1)` triggers an initialization error.

4. **Algorithmic & Math Justification:** Regularizes the model to prevent overfitting by breaking co-adaptation between hidden units, forcing the network to learn more robust feature representations.

Line 5: `LSTM(50, return_sequences=False),`

1. **Underlying Mechanics & Arguments:** Adds a second LSTM layer with 50 hidden units. Setting `return_sequences=False` instructs the layer to return only its final calculated hidden state vector (h_{30}) after processing the entire sequence.

2. Structural & Shape Transformation:

- Input tensor shape: `(None, 30, 50)` .
- Output tensor shape: `(None, 50)` . This collapses the time dimension, converting the 3D sequence tensor into a dense 2D matrix.

3. **Behavioral Alternatives:** Setting `return_sequences=True` preserves the time dimension, outputting a `(None, 30, 50)` tensor. This will cause an error when passed to the downstream `Dense` layer unless a flattening layer is added first.

4. **Algorithmic & Math Justification:** Condenses the extracted temporal sequence information into a fixed-length feature vector, preparing it for final linear projection layers.

Line 6: `Dropout(0.2),`

1. **Underlying Mechanics & Arguments:** Applies a 20% dropout regularizer to the 2D feature matrix output by the second LSTM layer.

2. Structural & Shape Transformation:

- Input tensor shape: `(None, 50)` .
- Output tensor shape: `(None, 50)` . Randomly zeroes out 20% of the feature vector elements during training passes.

3. **Behavioral Alternatives:** Removing this layer reduces the model's regularizing constraints, which can lead to overfitting if the hidden layers start memorizing the training sequence noise.

4. **Algorithmic & Math Justification:** Regularizes the condensed temporal feature vector before it is passed to the fully connected dense layers.

Line 8: Dense(25),

1. **Underlying Mechanics & Arguments:** Adds a fully connected (dense) layer with 25 neurons. Since no activation function is specified, it defaults to a linear projection ($f(x) = Wx + b$).
2. **Structural & Shape Transformation:**
 - Input tensor shape: (None, 50) .
 - Output tensor shape: (None, 25) . It performs a matrix multiplication of the input by a weight matrix $W \in \mathbb{R}^{50 \times 25}$ and adds a bias vector $b \in \mathbb{R}^{25}$.
3. **Behavioral Alternatives:** Adding a non-linear activation function like `activation='relu'` changes the layer to output $\max(0, Wx + b)$, which introduces non-linear feature mapping before the final output prediction.
4. **Algorithmic & Math Justification:** Reduces the hidden feature space from 50 to 25 dimensions, allowing the model to combine structural features before making the final prediction.

Line 9: Dense(1)

1. **Underlying Mechanics & Arguments:** The final fully connected layer containing 1 unit with a default linear activation function, acting as the continuous regression output head.
2. **Structural & Shape Transformation:**
 - Input tensor shape: (None, 25) .
 - Output tensor shape: (None, 1) . Multiplies the input by a weight matrix $W \in \mathbb{R}^{25 \times 1}$ and adds a bias scalar $b \in \mathbb{R}^1$.
3. **Behavioral Alternatives:** Using a bounded activation function like `activation='sigmoid'` limits the output values to the range $(0, 1)$, which will prevent the model from predicting values outside that range if the target scale changes.
4. **Algorithmic & Math Justification:** Projects the internal features down to a single continuous value, matching the shape of the target regression labels.

Line 10:])

1. **Underlying Mechanics & Arguments:** Closes the layer initialization list array, completing the internal setup of the sequential execution graph structure.
2. **Structural & Shape Transformation:** finalizes layer registry sequences into the model pipeline configuration.
3. **Behavioral Alternatives:** Syntax mismatch errors will occur if closing brackets are deleted or misaligned.

4. **Algorithmic & Math Justification:** Finalizes structural layer assembly, preparing the network graph for compilation.

Line 12: `model.compile(optimizer='adam', loss='mean_squared_error')`

1. **Underlying Mechanics & Arguments:** Configures the model's learning settings by binding the training algorithms:
 - `optimizer='adam'` : Selects the Adam optimization algorithm. Adam computes adaptive learning rates for each parameter by tracking first-moment (mean) and second-moment (uncentered variance) gradients.
 - `loss='mean_squared_error'` : Selects Mean Squared Error (MSE) as the objective loss function to be minimized during training.
2. **Structural & Shape Transformation:** Allocates optimization states and loss calculation trackers in memory. The model's weight matrices change state from uncompiled definitions to trainable mathematical structures.
3. **Behavioral Alternatives:** Choosing a classification loss function like `loss='categorical_crossentropy'` will cause runtime evaluation errors during training because the target values are continuous floats, not discrete class labels.
4. **Algorithmic & Math Justification:** Pairs the model with an appropriate optimizer and loss function for regression tasks. Minimizing the MSE loss directly penalizes larger prediction errors, forcing the model to fit the underlying time series trend:
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Line 14: `model.summary()`

1. **Underlying Mechanics & Arguments:** Diagnostic print utility that displays the structural layout of the network, including layer types, output shapes, and parameter counts, to the console.
 2. **Structural & Shape Transformation:** Diagnostic step; outputs structural model data to `stdout`.
 3. **Behavioral Alternatives:** Omitting this statement suppresses the structural diagnostic summary but has no effect on model training or underlying tensor transformations.
 4. **Algorithmic & Math Justification:** Provides visual validation of layer structural transformations, confirming that shapes match across layers and that parameter totals align with expectations.
-

Cell 4: Model Training Pipeline

Line 1: `history = model.fit()`

1. **Underlying Mechanics & Arguments:** Triggers the model's training loop. It executes forward passes, loss calculations, backpropagation through time (BPTT), and weight optimization updates, storing the resulting training metrics in a `History` object.

2. **Structural & Shape Transformation:** Updates the internal model weight parameters across training cycles, outputting a state tracking history object reference.
3. **Behavioral Alternatives:** Calling training runs without capturing the output variable (`model.fit(...)`) trains the weights normally but drops historical metric records, making it difficult to plot or analyze convergence curves.
4. **Algorithmic & Math Justification:** Orchestrates the internal learning routines that update network parameters to minimize prediction error.

Line 2: `X_train,`

1. **Underlying Mechanics & Arguments:** Passes the 3D training feature tensor as the primary input matrix for model optimization.
2. **Structural & Shape Transformation:** Input array of shape `(776, 30, 1)` with data type `float64` is converted into batches of TensorFlow tensors.
3. **Behavioral Alternatives:** Passing an incorrect data structure (like an un-indexed dictionary) causes shape parsing failures and halts execution with an initialization error.
4. **Algorithmic & Math Justification:** Supplies the historical sequence data used to update the model's weights during training.

Line 3: `y_train,`

1. **Underlying Mechanics & Arguments:** Passes the continuous target training labels used to calculate prediction errors.
2. **Structural & Shape Transformation:** Input matrix of shape `(776, 1)` with data type `float64` is split into tensor blocks matching the input features.
3. **Behavioral Alternatives:** Mismatching the sample length of targets relative to features triggers a structural dimension mismatch error.
4. **Algorithmic & Math Justification:** Provides the ground-truth targets required to calculate loss and generate gradients during supervised learning.

Line 4: `batch_size=32,`

1. **Underlying Mechanics & Arguments:** Sets the mini-batch size to 32. The training routine divides the dataset into blocks of 32 samples, calculates gradients over each block, and updates model weights after each batch.
2. **Structural & Shape Transformation:** Slices the input training tensor into sub-batches of shape `(32, 30, 1)` and corresponding label matrices of shape `(32, 1)` for graph execution.
3. **Behavioral Alternatives:** Setting `batch_size=len(X_train)` switches the optimizer to full-batch gradient descent, which increases memory usage and slows down parameter convergence.

4. **Algorithmic & Math Justification:** Mini-batching balances memory efficiency and gradient stability, providing a regularizing effect that helps the optimizer escape local minima.

Line 5: `epochs=20,`

1. **Underlying Mechanics & Arguments:** Specifies that the training loop will perform 20 complete iterative passes over the entire training dataset.
2. **Structural & Shape Transformation:** Executes the optimization routines across 20 training cycles, running 22 gradient update steps per epoch ($\lceil 698/32 \rceil = 22$, accounting for the validation split).
3. **Behavioral Alternatives:** Setting `epochs=1` stops training before the weights have fully converged, resulting in an underfitted model. Setting it to `1000` without early stopping can lead to overfitting as the model starts memorizing noise.
4. **Algorithmic & Math Justification:** Controls the training duration, ensuring the model has enough optimization cycles to minimize loss without overfitting to the training samples.

Line 6: `validation_split=0.1,`

1. **Underlying Mechanics & Arguments:** Reserves the final 10% of the training dataset ($776 \times 0.1 \approx 78$ samples) to serve as an internal validation set. The model monitors its performance on this holdout data at the end of each epoch to track generalization.
2. **Structural & Shape Transformation:**
 - Active training set shape: $(698, 30, 1)$.
 - Active validation set shape: $(78, 30, 1)$.
3. **Behavioral Alternatives:** Setting `validation_split=0.0` disables validation tracking, meaning the model will only log training loss and won't provide feedback on how well it generalizes during training.
4. **Algorithmic & Math Justification:** Provides an unbiased estimate of model performance on unseen data during training, helping detect overfitting early if validation loss begins to diverge from training loss.

Line 7: `shuffle=False`

1. **Underlying Mechanics & Arguments:** Disables shuffling, ensuring that mini-batches are extracted in strict chronological order during training passes.
2. **Structural & Shape Transformation:** Preserves the existing chronological order of sample sequences in memory.
3. **Behavioral Alternatives:** Setting `shuffle=True` randomizes the sequence positions across mini-batches. While acceptable for sliding window setups where sequences are already formed, keeping it `False` helps maintain contiguous time-series states across training updates.

4. **Algorithmic & Math Justification:** Disabling shuffling ensures that consecutive time-series segments are processed in chronological order, which can be beneficial for maintaining state stability in recurrent networks.

Line 8:)

1. **Underlying Mechanics & Arguments:** Closes the arguments list of the `.fit()` method, executing the compilation graph training operations.
 2. **Structural & Shape Transformation:** Triggers training execution, shifting model internal weights across steps.
 3. **Behavioral Alternatives:** Syntax parsing errors will occur if closing bracket components are missing.
 4. **Algorithmic & Math Justification:** Marks the end of parameter configurations, kicking off the optimization process.
-

Cell 5: Evaluation, Prediction & Visualization

Line 1: `predictions = model.predict(X_test)`

1. **Underlying Mechanics & Arguments:** Runs a forward pass through the trained model using the testing features `X_test`, disabling dropout regularizers to generate deterministic inference outputs.
2. **Structural & Shape Transformation:**
 - Input tensor shape: `(194, 30, 1)`.
 - Output tensor shape: `(194, 1)` with data type `float32`, containing normalized predictions.
3. **Behavioral Alternatives:** Passing a tensor with incorrect dimensions (such as a 2D array instead of a 3D sequence) will fail shape validation and throw a `ValueError`.
4. **Algorithmic & Math Justification:** Generates model forecasts on unseen evaluation data to verify the network's predictive accuracy.

Line 3: `predictions = scaler.inverse_transform(predictions)`

1. **Underlying Mechanics & Arguments:** Uses the trained scaler's `inverse_transform` method to reverse the min-max normalization, converting predictions back to their original physical scale (degrees Celsius).
2. **Structural & Shape Transformation:**
 - Input array shape: `(194, 1)` with data type `float32` (scaled between 0 and 1).
 - Output array shape: `(194, 1)` with data type `float64`, restored to original data scales.

3. **Behavioral Alternatives:** Skipping this step leaves predictions scaled between 0 and 1, creating a scale mismatch when comparing them against unscaled target temperatures.
4. **Algorithmic & Math Justification:** Reverses the feature scaling transformation so that model outputs can be evaluated and interpreted in their original unit dimensions: $x_{\text{original}} = x_{\text{scaled}} \cdot (x_{\text{max}} - x_{\text{min}}) + x_{\text{min}}$

Line 4: `y_test_unscaled = scaler.inverse_transform(y_test.reshape(-1, 1))`

1. **Underlying Mechanics & Arguments:** Reshapes the 2D testing labels to ensure correct alignment before reversing the min-max scaling, restoring the ground-truth values to their original physical units.
2. **Structural & Shape Transformation:**
 - Input `y_test` shape: (194, 1).
 - Output `y_test_unscaled` shape: (194, 1) with data type `float64` , containing unscaled target values.
3. **Behavioral Alternatives:** Passing an un-reshaped 1D target vector can trigger dimension errors if the scaler expects a strict sample-by-feature matrix format.
4. **Algorithmic & Math Justification:** Converts test labels back to their original scale, providing an accurate baseline for performance evaluation and plotting.

Line 6: `plt.figure(figsize=(12, 6))`

1. **Underlying Mechanics & Arguments:** Instantiates a new figure plotting canvas window. The parameter `figsize=(12, 6)` defines the structural drawing width and height in inches.
2. **Structural & Shape Transformation:** Allocates a canvas window container tracking object in the active graphics backend.
3. **Behavioral Alternatives:** Shrinking size scales to `figsize=(2,1)` compresses graphics labels, making the plot unreadable. Omitting this step uses the default figure canvas size dimensions.
4. **Algorithmic & Math Justification:** Allocates a clean graphical canvas, adjusting dimensions to ensure that long time-series sequences are clearly visible.

Line 8: `plt.plot(y_test_unscaled, label='Actual Temperature')`

1. **Underlying Mechanics & Arguments:** Plots the unscaled ground-truth test values as a continuous line on the active canvas, assigning it the string identifier `'Actual Temperature'` .
2. **Structural & Shape Transformation:** Maps the 1D data sequence values onto graphical coordinates on the active figure plot.
3. **Behavioral Alternatives:** Passing data with mismatched dimensions will raise a plotting layout error. Changing the format string alters how the data lines are styled.

4. **Algorithmic & Math Justification:** Renders the historical ground-truth timeline, establishing the base baseline for visual comparison.

Line 9: `plt.plot(predictions, label='Predicted Temperature')`

1. **Underlying Mechanics & Arguments:** Plots the unscaled model predictions as a continuous line on the same canvas, labeling it 'Predicted Temperature' .
2. **Structural & Shape Transformation:** Maps the prediction vector values onto the active figure canvas plot layout.
3. **Behavioral Alternatives:** Slicing or shifting the array indices beforehand will misalign the predictions relative to the test targets, distorting the temporal comparison.
4. **Algorithmic & Math Justification:** Overlays the model's predictions directly onto the ground-truth line, allowing for quick visual analysis of forecast errors and phase alignments.

Line 11: `plt.title('Weather Prediction (LSTM)')`

1. **Underlying Mechanics & Arguments:** Appends the specified descriptive text title string to the top region axis of the active figure canvas.
2. **Structural & Shape Transformation:** No data array changes; updates text properties on the active plot layout.
3. **Behavioral Alternatives:** Omitting this leaves the plot untitled, which reduces clarity but has no impact on data execution loops.
4. **Algorithmic & Math Justification:** Adds context to the visualization, identifying the model architecture and dataset being evaluated.

Line 12: `plt.xlabel('Days')`

1. **Underlying Mechanics & Arguments:** Binds the descriptive text label string 'Days' to the horizontal X-axis of the active figure plot.
2. **Structural & Shape Transformation:** No data array changes; updates text properties on the active plot layout.
3. **Behavioral Alternatives:** Changing the string or omitting it leaves the axis unlabeled, reducing the context of the timeline increments.
4. **Algorithmic & Math Justification:** Labels the independent variable axis, defining the unit steps (days) for the time-series visualization.

Line 13: `plt.ylabel('Temperature')`

1. **Underlying Mechanics & Arguments:** Binds the descriptive text label string 'Temperature' to the vertical Y-axis of the active figure plot.

2. **Structural & Shape Transformation:** No data array changes; updates text properties on the active plot layout.
3. **Behavioral Alternatives:** Omitting the label removes context regarding the physical unit scales being measured.
4. **Algorithmic & Math Justification:** Labels the dependent variable axis, confirming that the data values are displayed in their original physical units.

Line 15: `plt.legend()`

1. **Underlying Mechanics & Arguments:** Compiles the tracking label strings defined in previous plotting commands and renders an informational legend box on the active canvas.
2. **Structural & Shape Transformation:** No data array changes; adds a descriptive overlay box to the figure plot.
3. **Behavioral Alternatives:** Skipping this step suppresses the legend display, making it difficult to distinguish between the actual and predicted data lines.
4. **Algorithmic & Math Justification:** Provides critical context for the visualization, allowing viewers to easily identify and compare ground-truth values against model predictions.

Line 16: `plt.show()`

1. **Underlying Mechanics & Arguments:** Flushes the graphic rendering pipeline, displays the final compiled figure plot window on screen, and releases internal engine memory allocation tracking pointers once closed.
2. **Structural & Shape Transformation:** Finalizes the visualization process, flushing graphic asset components out of active runtime memory.
3. **Behavioral Alternatives:** In non-interactive environments, failing to call this method can prevent the plot from rendering entirely, leaving it hidden in memory buffers.
4. **Algorithmic & Math Justification:** Completes the analysis pipeline, rendering the final plot to evaluate the LSTM model's tracking performance on seasonal weather fluctuations.